

Draft version
Not for distribution without
authors' permission

Table of Contents

	Page
1. Introduction	1
2. The basic formulation of RNN	1
2.1 Forward pass and loss calculations	1
2.2 Backpropagation Through Time (BPTT)	8
3. PyTorch Implementation of RNN to Approximate a Hypothetical Reservoir Flood Routing	16
3.1 RNN as surrogate model for level-pool routing	16
3.2 Step-by-step implementation	18
Reference	36
Appendix	37

1. Introduction

So far, in the previous modules, we haven't encountered and dealt with time-dependent or sequential data. In real-world scenarios, there are many situations where the current data relies on the values and behaviour of the previous ones.

In engineering, this can be found in dynamics or transient analysis where the present solution may depend on the solutions at the previous time steps.

In Natural Language Processing (NLP), the prediction of the next word depends on the previous words and their order.

Similarly, in stock market prediction, historical stock prices and their dynamics over the previous days, weeks, months or even years may provide important information for predicting their future behaviour.

Such sequential and time-dependent phenomenon will benefit from a neural network that can consider the previous history of the data leading to its present state. In other words, a network that can "remember" or having some "memory" of past information would be very suitable for such sequential problems.

2. The basic formulation of RNN

As a neural network, RNN consists of forward pass and backward pass procedures. The former is used to produce the predicted outputs for the calculation of the loss function, while the latter is used to calculate the gradients of the loss function with respect to the network parameters. These gradients are subsequently used by the optimizer to update the parameters.

In this module, like in the previous modules, we will deal with a regression problem and in the later application section, we will detail the RNN PyTorch implementation for flood routing.

2.1 Forward pass and loss calculations

Consider Figure 1. It shows a typical input and output distribution of a time-dependent system. It can represent, for example, electrical signals or mechanical responses. For our discussion, we take the figure as showing typical reservoir inflow and outflow hydrographs.

Figure 2 shows the discretization of the hydrographs into 10 time steps with constant time interval, Δ_t . The discretized values of the inflow and outflow hydrographs can be represented by the vectors Inflow, $\{I\}$, and Outflow, $\{O\}$, respectively. The corresponding time steps are represented by the vector, $\{t\}$.

$$\text{Inflow, } \{I\} = \begin{Bmatrix} I_1 \\ I_2 \\ I_3 \\ \vdots \\ \vdots \\ I_{10} \end{Bmatrix}, \text{ Outflow, } \{O\} = \begin{Bmatrix} O_1 \\ O_2 \\ O_3 \\ \vdots \\ \vdots \\ O_{10} \end{Bmatrix}, \text{ Time step, } \{t\} = \begin{Bmatrix} t_1 \\ t_2 \\ t_3 \\ \vdots \\ \vdots \\ t_{10} \end{Bmatrix} \quad (1)$$

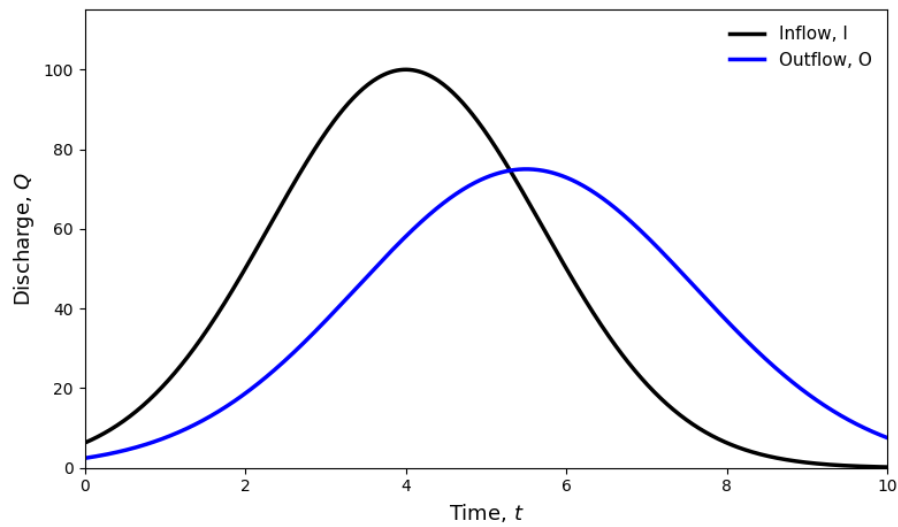


Figure 1: Typical reservoir inflow and outflow hydrographs

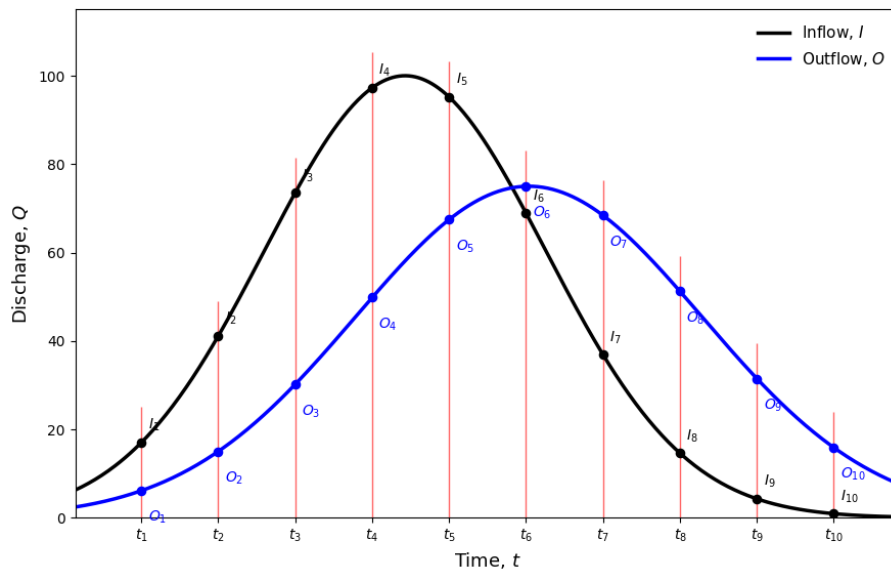


Figure 2: Discretization of inflow and outflow hydrographs

2.1.1 Establish the parameter matrices and vectors

Just like the kernel in CNN (Module 3), we decide on the number of parameters (i.e., weight and biases) by deciding on the size of the corresponding matrices and vectors. For our discussion, we consider an RNN cell of two neurons, with single/scalar input. Thus, we have:

Weights matrix, $[W]$ and weights vectors, $\{u\}$ and $\{v\}$

$$[W] = \begin{bmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \end{bmatrix}$$

(2a)

$$\begin{aligned}\{u\} &= \begin{Bmatrix} u_1 \\ u_2 \end{Bmatrix} \\ \{v\} &= \begin{Bmatrix} v_1 \\ v_2 \end{Bmatrix}\end{aligned}$$

Biases vector, $\{b\}$ and bias scalar, c

$$\begin{aligned}\{b\} &= \begin{Bmatrix} b_1 \\ b_2 \end{Bmatrix} \\ c &\end{aligned} \tag{2b}$$

The weights in $[W]$ will multiply the hidden-state vector, $\{h\}$, the weights in $\{u\}$ will multiply the input, and the weights in $\{v\}$ will multiply the hidden-state vector after activation, to obtain the output.

The biases in $\{b\}$ will be added in calculating the hidden-state vector, while c will be added to the $\{h\}^T \{v\}$ in calculating the output.

The roles and locations of these matrices and vectors, including what has been mentioned as hidden-state vector, $\{h\}$, will be made clearer once we discuss the network of RNN, which is next.

2.1.2 RNN network unfolded in time

To establish the RNN network, we will discuss it based on the progression of time steps. We will elaborate the first two time steps to make obvious the evolution through time, and then jump to the 10th time step of the discretized data given by Eq. (1).

1st time step (i.e. t_1)

Consider Figure 3. It shows the RNN network at the first time step (i.e., t_1). To highlight the time step, say t_i , for double-indexed time-dependent variables such as $\{z_i\} = \begin{Bmatrix} z_{i,1} \\ z_{i,2} \end{Bmatrix}$, and $\{h_i\} = \begin{Bmatrix} h_{i,1} \\ h_{i,2} \end{Bmatrix}$, the first index of the subscript, i , represents the time step, while the second index, j , represents the location in the vector. Thus, the components can generally be written as $z_{i,j}$ and $h_{i,j}$. For single-indexed time-dependent variables such as the input and output, they are simply written as I_i and $\hat{O}_i$, respectively.

The box is referred to as the RNN cell. It has two processes, one, to sum all variables coming into the cell, represented by the summing vector $\{z_1\} = \begin{Bmatrix} z_{1,1} \\ z_{1,2} \end{Bmatrix}$ and two, to activate the output from the cell, which is called the hidden-state vector $\{h_1\} = \begin{Bmatrix} h_{1,1} \\ h_{1,2} \end{Bmatrix}$.

Notice that these two processes are quite similar to the FFNN discussed in Module 1 and 2. The only difference is that, besides the product of the input and the weights (i.e., $\{U\}I_1$) and the biases

$\{b\}$, the summing vector $\{z_1\}$ would also consist of the product of the weight matrix, $[W]$ and the hidden-state vector from the previous time step (i.e., $[W]\{h_0\}$), thus:

$$\{z_1\} = \begin{Bmatrix} z_{1,1} \\ z_{1,2} \end{Bmatrix} = \begin{Bmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \end{Bmatrix} \begin{Bmatrix} h_{0,1} \\ h_{0,2} \end{Bmatrix} + \begin{Bmatrix} u_1 \\ u_2 \end{Bmatrix} I_1 + \begin{Bmatrix} b_1 \\ b_2 \end{Bmatrix} \quad (3)$$

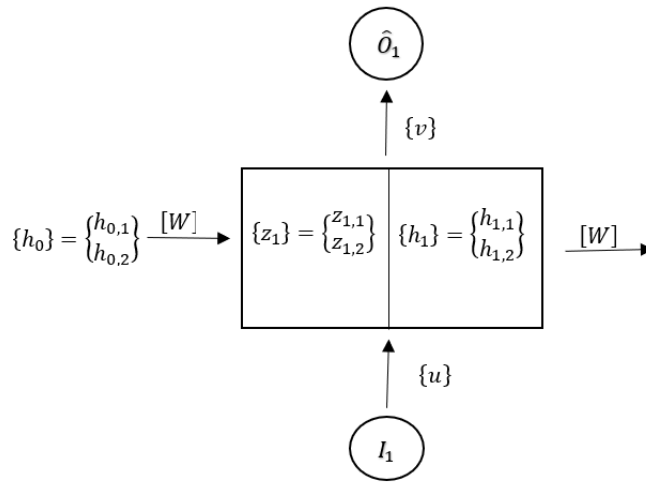


Figure 3: RNN network at 1st time step, t_1

Now, also similar to the FFNN, we need to activate Eq. (3) by an activation function. Such activation produces the hidden-state vector $\{h_1\}$. We can use various activations functions such as tanh, ReLU, GeLU etc., but for our discussion, we will use tanh. Thus:

$$\{h_1\} = \begin{Bmatrix} h_{1,1} \\ h_{1,2} \end{Bmatrix} = \begin{Bmatrix} \tanh(z_{1,1}) \\ \tanh(z_{1,2}) \end{Bmatrix} \quad (4)$$

For this 1st time step, one might wonder, what would be the previous hidden-state vector, $\{h_0\}$, as in Eq. (3)? This is the initial hidden state, which can be taken as zeros or treated as learnable parameters. In our discussion, we take the former (i.e., $\{h_0\} = \begin{Bmatrix} h_{0,1} \\ h_{0,2} \end{Bmatrix} = \begin{Bmatrix} 0 \\ 0 \end{Bmatrix}$).

With the hidden state obtained from Eq. (4), we can calculate the predicted output, $\hat{o}_1$ as:

$$\hat{o}_1 = \{v\}^T \{h_1\} + c = \begin{Bmatrix} v_1 & v_2 \end{Bmatrix} \begin{Bmatrix} h_{1,1} \\ h_{1,2} \end{Bmatrix} + c \quad (5)$$

The error, E_1 and the corresponding squared-error loss, L_1 for this 1st time step are thus:

$$E_1 = o_1 - \hat{o}_1 \quad (6)$$

$$L_1 = (o_1 - \hat{o}_1)^2 \quad (7)$$

2nd time step (i.e. t_2)

To see the evolution through time, let's consider the 2nd time step. Figure 4 shows the network at t_2 . Notice that not much has changed except for the subscripts indicating that we are now at a different time step than before. Notice also that we are sharing the weights and biases, meaning that there are no time-dependent subscripts associated with these parameters. The subscripts associated with the weights and biases only indicate their locations in the corresponding matrices or vectors.

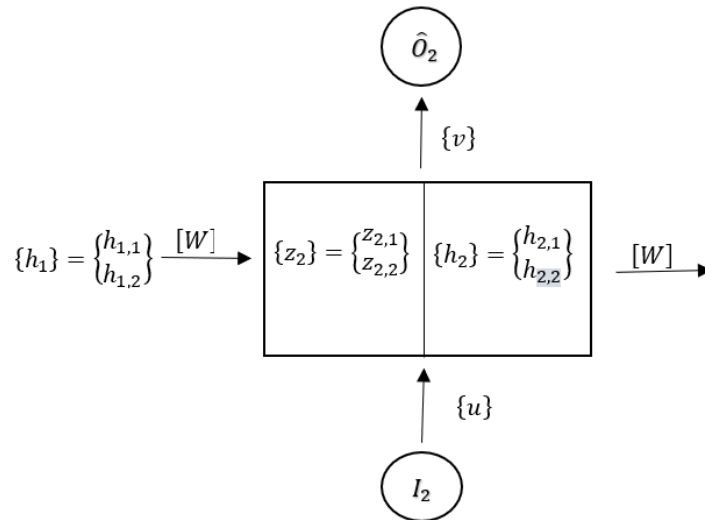


Figure 4: RNN network at 2nd time step, t_2

Now, the summing vector has become $\{z_2\}$:

$$\{z_2\} = \begin{Bmatrix} z_{2,1} \\ z_{2,2} \end{Bmatrix} = \begin{Bmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \end{Bmatrix} \begin{Bmatrix} h_{1,1} \\ h_{1,2} \end{Bmatrix} + \begin{Bmatrix} u_1 \\ u_2 \end{Bmatrix} I_2 + \begin{Bmatrix} b_1 \\ b_2 \end{Bmatrix} \quad (8)$$

Notice that $\begin{Bmatrix} h_{1,1} \\ h_{1,2} \end{Bmatrix}$, which is the hidden state from the 1st time step, is now carried forward to present time step. Thus, the calculation at t_2 depends not only on the present input, I_2 , but also on the hidden state from the previous time step, $\{h_1\}$. This provides the RNN with its ability to carry information from its “history”.

By activating Eq. (8) we then obtain the hidden-state vector at t_2 :

$$\{h_2\} = \begin{Bmatrix} h_{2,1} \\ h_{2,2} \end{Bmatrix} = \begin{Bmatrix} \tanh(z_{2,1}) \\ \tanh(z_{2,2}) \end{Bmatrix} \quad (9)$$

and the output, $\hat{O}_2$:

$$\hat{O}_2 = \{v\}^T \{h_2\} + c = \begin{Bmatrix} v_1 & v_2 \end{Bmatrix} \begin{Bmatrix} h_{2,1} \\ h_{2,2} \end{Bmatrix} + c \quad (10)$$

The error, E_2 and the corresponding squared-error loss, L_2 for this 2nd time step are thus:

$$E_2 = O_2 - \hat{O}_2 \quad (11)$$

$$L_2 = (O_2 - \hat{O}_2)^2 \quad (12)$$

10th time step (i.e. t_{10})

For completeness, let's consider the network at the final time step, t_{10} , as shown in Figure 5. Based on the figure, the summing vector has become $\{z_{10}\}$:

$$\{z_{10}\} = \begin{Bmatrix} z_{10,1} \\ z_{10,2} \end{Bmatrix} = \begin{Bmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \end{Bmatrix} \begin{Bmatrix} h_{9,1} \\ h_{9,2} \end{Bmatrix} + \begin{Bmatrix} u_1 \\ u_2 \end{Bmatrix} I_{10} + \begin{Bmatrix} b_1 \\ b_2 \end{Bmatrix} \quad (13)$$

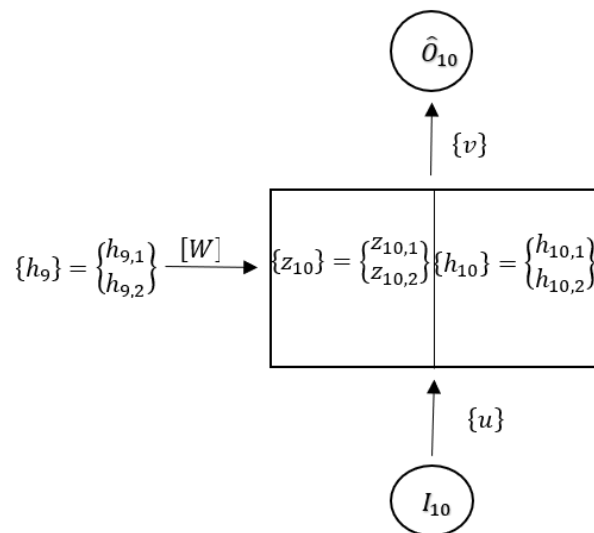


Figure 5: RNN network at 10th time step, t_{10}

Just notice that Eq. (13) consists of input at the 10th time step (i.e., I_{10}), the hidden state from the 9th time step (i.e., $\{h_{9,1}\}$), and the parameters (weights and biases) shared throughout the time steps. Also notice that there is no arrow on the right side of the box, indicating that t_{10} is the final time step considered in our discretized sequence.

By activating Eq. (13), we then obtain the hidden-state vector at t_{10} :

$$\{h_{10}\} = \begin{Bmatrix} h_{10,1} \\ h_{10,2} \end{Bmatrix} = \begin{Bmatrix} \tanh(z_{10,1}) \\ \tanh(z_{10,2}) \end{Bmatrix} \quad (14)$$

and subsequently, the predicted output, $\hat{O}_{10}$:

$$\hat{O}_{10} = \{v\}^T \{h_{10}\} + c = \{v_1 \quad v_2\} \begin{Bmatrix} h_{10,1} \\ h_{10,2} \end{Bmatrix} + c \quad (15)$$

The error, E_{10} and the corresponding squared-error loss, L_{10} for this 10th time step are thus:

$$E_{10} = O_{10} - \hat{O}_{10} \quad (16)$$

$$L_{10} = (O_{10} - \hat{O}_{10})^2 \quad (17)$$

In Figure 3 to 5, the network is illustrated separately for each time step to ease the derivation and discussion. However, in literature, they can be combined (unfolded) or folded. The former representation is shown in Figure 6 while the latter is shown in Figure 7. Notice again that the same parameters, $[W]$, $\{u\}$ and $\{v\}$, are shared throughout all the time steps.

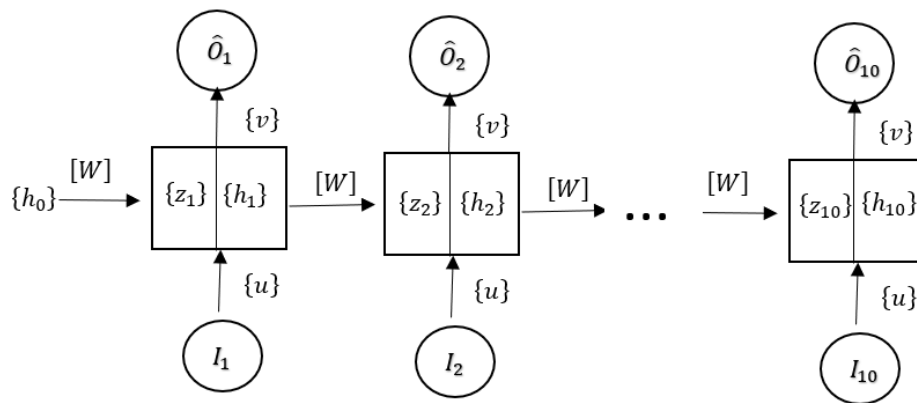


Figure 6: Unfolded representation of RNN

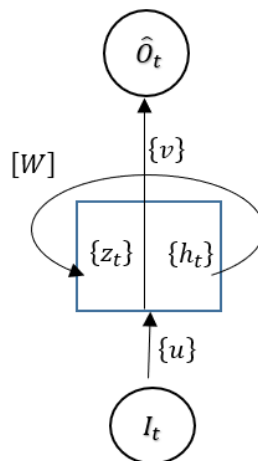


Figure 7: Folded representation of RNN

2.1.3 The total loss (the loss function of the network)

For the backpropagation and updating, gradients of the loss function with respect to the parameters are required. The loss function is contributed by the losses from all the time steps and can be calculated in various forms (i.e., MSE, MAE, Huber etc.). In our discussion, we opt for MSE. Thus, the loss function, L , is given as:

$$\begin{aligned} \text{Loss function (MSE), } L &= \frac{E_1^2 + E_2^2 + \dots + E_{10}^2}{10} \\ &= \frac{L_1 + L_2 + \dots + L_3}{10} \\ &= \frac{\sum_{t=1}^{10} L_t}{10} \end{aligned} \quad (18)$$

2.2 Backpropagation Through Time (BPTT)

For RNN, due its architecture which allows information (hidden states) to flow through time, the backpropagation needs to account for such time dependency. The loss gradient must be calculated not only with respect to the parameters through the particular time step (direct path), but also through the previous time steps (recurrent paths). This backpropagation is known as Backpropagation Through Time (BPTT).

To demonstrate the process, let's consider a simpler network with 3 time steps and scalar weights, w , u and v , scalar biases, b and c as well as scalar input, I_t , as shown in Figure 8 below.

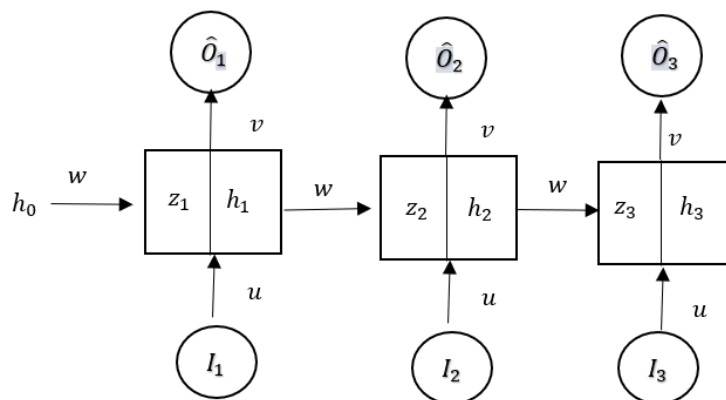


Figure 8: 3-time steps RNN with scalar parameters and input

Why scalar parameters, why not stick with matrices and vectors? Why 3 time steps, why not 2? First of all, in this discussion, we just want to highlight the basic concept of BPTT, as the “heavy lifting” will later be carried out by PyTorch. Thus, we want to keep things simple and traceable. Scalar states and parameters are therefore chosen as they simplify the derivation considerably, while their extension to vectors and matrices is straightforward.

On the number of time steps, 3 is chosen because it is the minimum number of steps that allows us to clearly demonstrate the compounding effect through repeated multiplication along the

recurrent path. As the number of time steps increases, such repeated multiplications of the recurrent weights and activation derivatives may lead to numerical issues known as vanishing or exploding gradients. It is important to understand the root cause of these numerical issues, as only then can we understand how they can be controlled and treated.

The errors for the network can be calculated as:

$$\begin{aligned}\text{Error at } t_1, E_1 &= (O_1 - \hat{O}_1) \\ \text{Error at } t_2, E_2 &= (O_2 - \hat{O}_2) \\ \text{Error at } t_3, E_3 &= (O_3 - \hat{O}_3)\end{aligned}\tag{19}$$

The squared-error losses for the network can then be calculated as:

$$\begin{aligned}\text{Loss at } t_1, L_1 &= E_1^2 = (O_1 - \hat{O}_1)^2 \\ \text{Loss at } t_2, L_2 &= E_2^2 = (O_2 - \hat{O}_2)^2 \\ \text{Loss at } t_3, L_3 &= E_3^2 = (O_3 - \hat{O}_3)^2\end{aligned}\tag{20}$$

The loss function for the network can be calculated as (MSE):

$$\text{Loss function, } L = \frac{L_1 + L_2 + L_3}{3}\tag{21}$$

2.2.1 Gradient descent and updating

In our discussion, to update the parameters we employ gradient descent:

$$\begin{aligned}u_{new} &= u_{old} - \mu \frac{\partial L}{\partial u} \\ v_{new} &= v_{old} - \mu \frac{\partial L}{\partial v} \\ w_{new} &= w_{old} - \mu \frac{\partial L}{\partial w} \\ b_{new} &= b_{old} - \mu \frac{\partial L}{\partial b} \\ c_{new} &= c_{old} - \mu \frac{\partial L}{\partial c}\end{aligned}\tag{22}$$

where μ is the learning rate. Based on Eq. (22), we can see that, to update the parameters, we need to calculate the loss gradients with respect to each of the parameters (i.e., $\frac{\partial L}{\partial u}, \frac{\partial L}{\partial v}, \frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}, \frac{\partial L}{\partial c}$).

However, herein, we will demonstrate the calculation of $\frac{\partial L}{\partial u}$ only, as it provides a clear demonstration of both the direct and recurrent paths through the three time steps.

i. Calculation of $\frac{\partial L}{\partial u}$

From Eq. (21):

$$\frac{\partial L}{\partial u} = \frac{1}{3} \left(\frac{\partial L_1}{\partial u} + \frac{\partial L_2}{\partial u} + \frac{\partial L_3}{\partial u} \right) \quad (23)$$

Now, let's calculate each of the gradients in the parentheses in Eq. (23).

ii. Calculation of $\frac{\partial L_1}{\partial u}$

$\frac{\partial L_1}{\partial u}$ is the gradient of the loss at the 1st time step (i.e., t_1) with respect to u . Figure 9 shows the backward pass for this gradient. As can be seen, there is only a direct path from the output, $\hat{O}_1$ back to u (red-arrowed path).

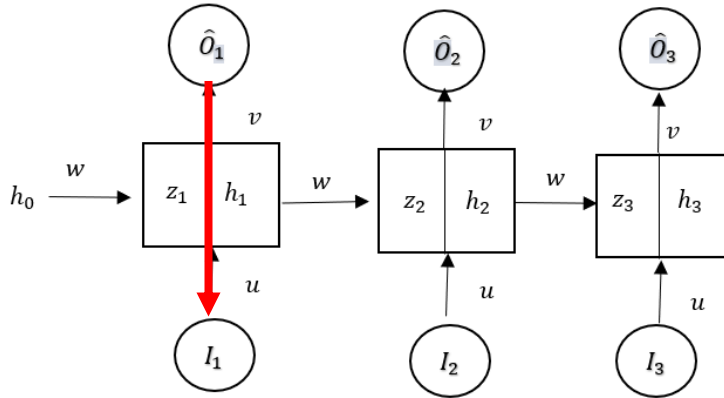


Figure 9: Direct path for the calculation of $\frac{\partial L_1}{\partial u}$

Therefore:

$$\begin{aligned} \frac{\partial L_1}{\partial u} &= \text{direct path at } t_1 \\ &= \frac{\partial L_1}{\partial \hat{O}_1} \frac{\partial \hat{O}_1}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial u} \end{aligned} \quad (24)$$

Notice that each derivative in Eq. (24) can be further calculated using the standard differentiation and chain-rule procedures. However, in this discussion, we are more interested in showing how the gradient paths evolve through time, which is the main concept behind BPTT. Thus, we will leave the derivatives in this form and let PyTorch do the “heavy lifting” later in the implementation.

iii. Calculation of $\frac{\partial L_2}{\partial u}$

$\frac{\partial L_2}{\partial u}$ is the gradient of the loss at the 2nd time step (i.e., t_2) with respect to u . Figure 10 shows the backward pass for this gradient. As can be seen, now, there are two paths, a direct path from the output, $\hat{O}_2$, to u at the 2nd time step (red-arrowed path) and a recurrent path from $\hat{O}_2$ to u at the 1st time step (blue-arrowed path). Now, we are witnessing for the first time the backward pass through time. Thus:

$$\begin{aligned} \frac{\partial L_2}{\partial u} &= \text{direct path at } t_2 + \text{recurrent path back to } t_1 \\ &= \frac{\partial L_2}{\partial \hat{O}_2} \frac{\partial \hat{O}_2}{\partial h_2} \frac{\partial h_2}{\partial z_2} \frac{\partial z_2}{\partial u} + \frac{\partial L_2}{\partial \hat{O}_2} \frac{\partial \hat{O}_2}{\partial h_2} \frac{\partial h_2}{\partial z_2} \underbrace{\frac{\partial z_2}{\partial h_1}}_{\text{Temporal bridge}} \frac{\partial h_1}{\partial u} \end{aligned} \quad (25)$$

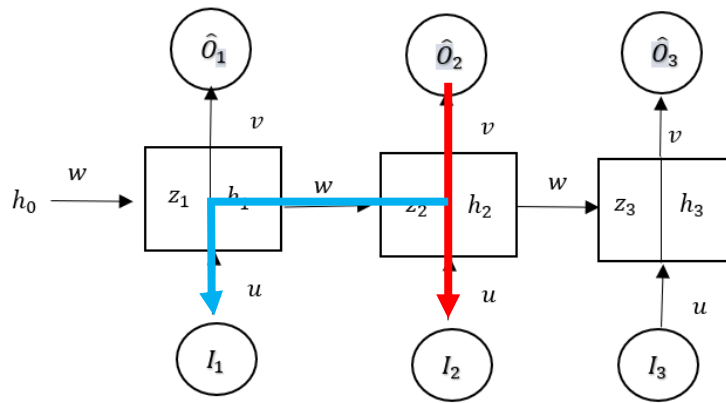


Figure 10: Direct and recurrent paths for the calculation of $\frac{\partial L_2}{\partial u}$

As can be seen in Eq. (25), in the recurrent path, there exists the term “temporal bridge”. It represents the influence of the previous time step on the present one. It is called a bridge because it connects the gradients across time. It is the key that allows RNN to consider the effects of the history on the present time, but the repeated multiplication along these temporal bridges can also be the root cause of numerical issues (vanishing or exploding gradients) if the backward flow becomes too long. The latter can be shown mathematically if we consider the 3rd time step, as this would be the minimum number of time steps for the recurrent weight to be multiplied by itself (as well as for the repeated multiplication of the activation function derivatives) to appear. So, let’s proceed with the final time step (i.e., the 3rd time step).

iv. Calculation of $\frac{\partial L_3}{\partial u}$

Figure 11 shows the backward pass for $\frac{\partial L_3}{\partial u}$. As can be seen the recurrent path (blue-arrowed paths) branches to u at the 1st and the 2nd time steps.

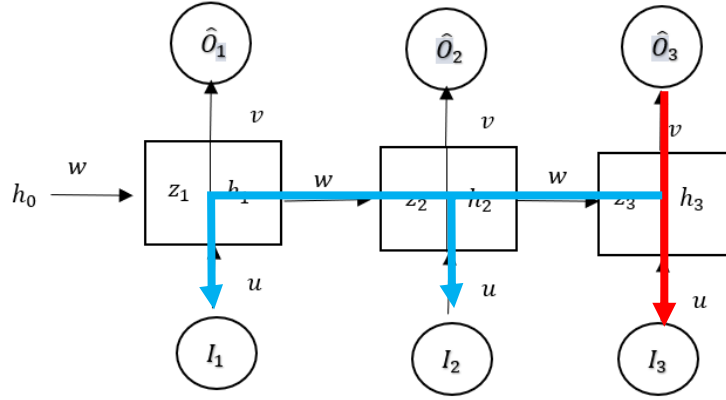


Figure 11: Direct and recurrent paths for the calculation of $\frac{\partial L_3}{\partial u}$

As a result, we have:

$$\begin{aligned} \frac{\partial L_3}{\partial u} &= \text{direct path at } t_3 + \text{recurrent path back to } t_2 + \text{recurrent path back to } t_1 \\ &= \frac{\partial L_3}{\partial \hat{O}_3} \frac{\partial \hat{O}_3}{\partial h_3} \frac{\partial h_3}{\partial z_3} \frac{\partial z_3}{\partial u} + \frac{\partial L_3}{\partial \hat{O}_3} \frac{\partial \hat{O}_3}{\partial h_3} \frac{\partial h_3}{\partial z_3} \underbrace{\frac{\partial z_3}{\partial h_2}}_{\text{Temporal bridge 1}} \frac{\partial h_2}{\partial u} + \frac{\partial L_3}{\partial \hat{O}_3} \frac{\partial \hat{O}_3}{\partial h_3} \frac{\partial h_3}{\partial z_3} \underbrace{\frac{\partial z_3}{\partial h_2}}_{\text{Temporal bridge 1}} \frac{\partial h_2}{\partial z_2} \underbrace{\frac{\partial z_2}{\partial h_1}}_{\text{Temporal bridge 2}} \frac{\partial h_1}{\partial u} \end{aligned} \quad (26)$$

Now, as can be seen, not only do we have more and longer terms of the chain rule (due to the branching of the recurrent paths), but we also have two temporal bridges. We will discuss the temporal bridges and their connection to numerical stabilities, but let's complete first the total gradient, $\frac{\partial L}{\partial u}$, thus:

$$\begin{aligned} \frac{\partial L}{\partial u} &= \frac{\partial L_1}{\partial \hat{O}_1} \frac{\partial \hat{O}_1}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial u} + \frac{\partial L_2}{\partial \hat{O}_2} \frac{\partial \hat{O}_2}{\partial h_2} \frac{\partial h_2}{\partial z_2} \frac{\partial z_2}{\partial u} + \frac{\partial L_2}{\partial \hat{O}_2} \frac{\partial \hat{O}_2}{\partial h_2} \frac{\partial h_2}{\partial z_2} \frac{\partial z_2}{\partial h_1} \frac{\partial h_1}{\partial u} + \frac{\partial L_3}{\partial \hat{O}_3} \frac{\partial \hat{O}_3}{\partial h_3} \frac{\partial h_3}{\partial z_3} \frac{\partial z_3}{\partial u} + \\ &\quad \frac{\partial L_3}{\partial \hat{O}_3} \frac{\partial \hat{O}_3}{\partial h_3} \frac{\partial h_3}{\partial z_3} \frac{\partial z_3}{\partial h_2} \frac{\partial h_2}{\partial u} + \frac{\partial L_3}{\partial \hat{O}_3} \frac{\partial \hat{O}_3}{\partial h_3} \frac{\partial h_3}{\partial z_3} \frac{\partial z_3}{\partial h_2} \frac{\partial h_2}{\partial z_2} \frac{\partial z_2}{\partial h_1} \frac{\partial h_1}{\partial u} \end{aligned} \quad (27)$$

It's long, isn't it? And this is only for a network with scalar parameters and input and with 3 time steps. But don't worry, in practice, the "heavy lifting" will be automatically carried out by PyTorch. What we need to take home for this discussion is the basic idea of BPTT so that we would not use PyTorch or any other libraries as a total black box. And if in the future we are required to dig deeper into the chain-rule expansion, we can start from here.

2.2.2 Numerical instabilities (vanishing or exploding gradient)

Note: You can skip this section for now if the network is shallow and the problem does not require very long-term dependencies

Like any other numerical methods, NN, or RNN in particular, is also exposed to numerical instabilities that would adversely affect the training process and, consequently, the final

inference. While there are many numerical issues that need to be considered and treated, the ones which are immediate for our present discussion are the vanishing and exploding gradients.

Vanishing gradients occur when the calculated gradients become increasingly small as the recurrent paths flow further back in time. Eventually, the gradient contributions from the earlier time steps become so small that they are almost negligible. If the present state is highly influenced by an event much earlier in time (where the gradient has diminished), then the prediction will be off, despite the weights still being updated by the terms from the more recent time steps.

Exploding gradients, on the other hand, occur when the calculated gradients become increasingly large and eventually blow up, leading to unrealistically large changes in the weights or even numerical failure. The former may result in inaccurate inference, while the latter may simply cause the training process to fail (e.g., producing NaN/Inf values).

But, before we proceed, we must note that, while these numerical stabilities are real problems, for shallow networks and problems that do not require very long-term dependencies, they are generally less pronounced and can be treated, making RNN still an efficient architecture for sequential and dynamic analysis, provided that the shallow network is sufficient for learning the physical phenomenon.

So, what is the possible cause for these numerical stabilities? To answer this, we refer to the 3rd term on the RHS of Eq. (26), and we can see Temporal Bridges 1 and 2 simultaneously exist in that term. Let's reproduce the relevant terms together with their multipliers as below and carrying of the differentiation:

$$\frac{\partial h_3}{\partial z_3} \frac{\partial z_3}{\partial h_2} \frac{\partial h_2}{\partial z_2} \frac{\partial z_2}{\partial h_1} = (1 - \tanh^2(z_3)) \cdot w \cdot (1 - \tanh^2(z_2)) \cdot w \quad (28)$$

There are two observations we can make from Eq. (28). The first observation is the multiplication of w by itself (i.e., $w \cdot w$). And we can imagine this repeated multiplication if the number of time steps becomes large (imagine 100!). Now, if the magnitude w is less than 1 (i.e., $|w| < 1$), its repeated self-multiplication will tend to produce very small values as we move further back in time. On the other hand, if its magnitude is greater than 1 (i.e., $|w| > 1$), its repeated multiplication has the tendency to amplify the gradient. To note, for matrix-form weights (i.e., $[W]$), the corresponding measure is commonly associated with the spectral radius, $\rho([W]) = \max_i |\lambda_i|$, where λ_i are the eigenvalues of $[W]$.

The second observation is the repeated multiplication of the activation-function derivatives, (i.e. $(1 - \tanh^2(z_3)) \cdot (1 - \tanh^2(z_2))$). For the particular activation function, $\tanh(z)$, that we are using, each derivative has a value between 0 and 1, as shown in Figure 12. Thus, their repeated multiplication tends to make the gradient increasingly smaller. While this promotes the vanishing of gradients, it can somehow help to damp an exploding situation.

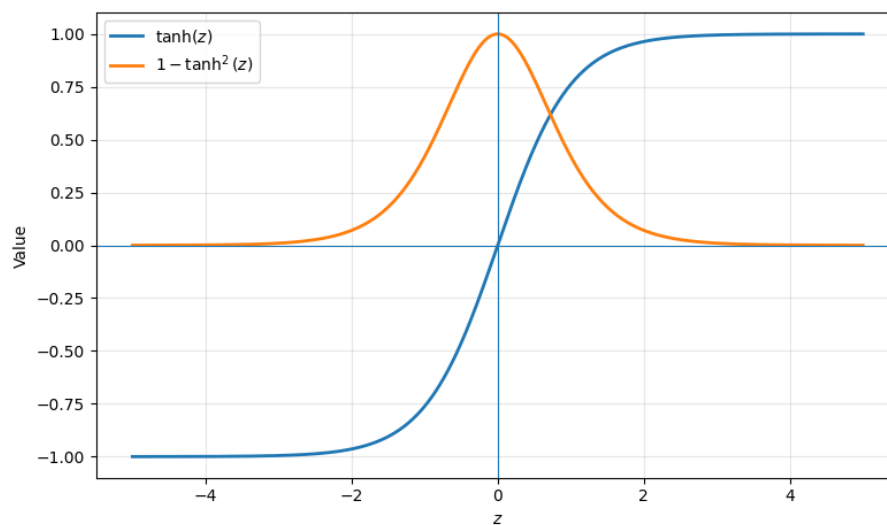


Figure 12: Tanh () activation function and its derivative

Making these two observations is important because not only do they allow us to identify the root cause of these numerical instabilities, but they also give us some clues on how they may be treated.

i. How to treat vanishing gradient

Based on our two observations, since one of the issues is related to the magnitude of the weights, the first thing to ensure is the proper initialization of the weights. Although we are not going to dwell on it, one way of ensuring such initialization is through an approach termed Orthogonal initialization, which can be automatically executed by PyTorch.

Secondly, try to avoid squashing activation functions such as $\tanh(z)$ and $\text{sigmoid}(z)$, as their derivatives can become very small when the activation functions approach their plateau regions. But to discuss this, we need to familiarize ourselves with the shape and derivatives of the various activation functions. Figure 13 shows the plot of $\text{sigmoid}(z)$, $\text{ReLU}(z)$ and Leaky $\text{ReLU}(z)$ and their derivatives, in addition to the plots of $\text{Tanh}()$ in Figure 12.

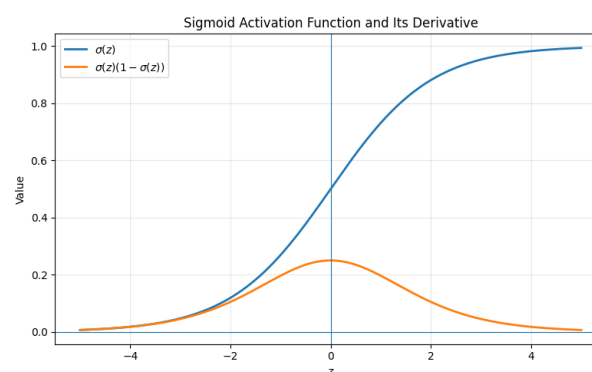


Figure 13: Various activation functions and their derivatives (continues)

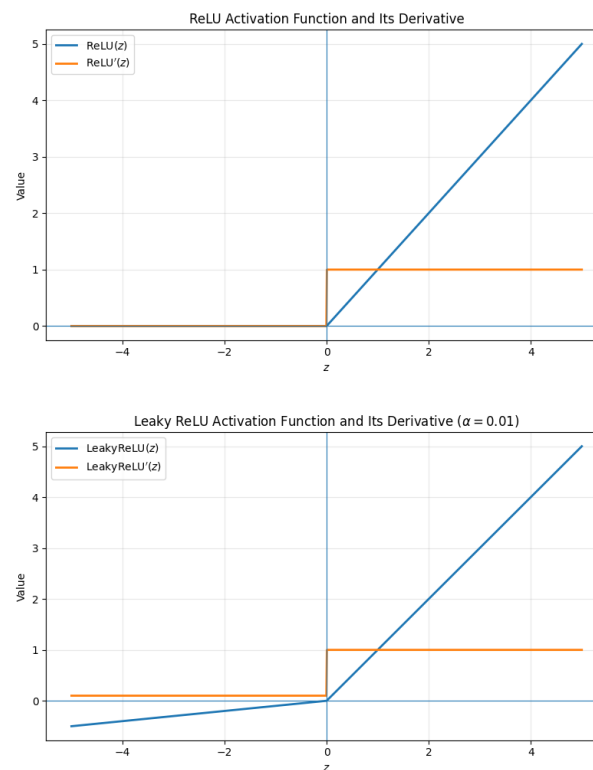


Figure 13: Various activation functions and their derivatives (continued)

As we have seen in Eq. (28) and as can be seen in Figure 12 and 13, the repeated multiplication of the small derivatives of $\tanh(z)$ and $\text{sigmoid}(z)$, can further promote the vanishing of gradients. A possible alternative is $\text{ReLU}(z)$, as its derivative is equal to 1 in its positive (linear) region. Thus, unlike $\tanh(z)$ and $\text{sigmoid}(z)$, it does not further diminish the gradient through the activation derivative when operating in this positive/linear region.

However, $\text{ReLU}(z)$ is also not a total solution. In its negative region, its derivative is zero. Even Leaky $\text{ReLU}(z)$ may not necessarily solve the problem because its small derivative in its negative region is still less than 1, and its repeated multiplication over many time steps can still cause the gradient to diminish.

Thus, what all this tells us is that changing the activation function may help to reduce the vanishing-gradient problem, but it does not necessarily solve it completely.

ii. How to treat exploding gradient

Exploding gradient occurs when the gradient values become very large. In this case, using squashing activation functions such as $\tanh()$ and $\text{Sigmoid}()$ might somehow help because their derivatives are bounded and can damp the gradient during the backward pass.

However, a more standard treatment is Gradient Clipping, where if the magnitude of the gradient exceeds a specified threshold, it is clipped or limited to prevent excessively large updates of the weights.

Another alternative is Weight Regularization (L1/L2), where we add a penalty to the loss function based on the size of the weights, with the intention of discouraging the network from learning excessively large weight values. This may indirectly help to reduce the tendency for the gradients to explode.

Of course, in PyTorch, such gradient clipping and regularization can be readily implemented..

iii. General treatment

A general treatment is to modify the plain vanilla RNN, leading to new architectures such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU). However, they deserve a module on their own.

But we must note that RNN itself is still efficient and is still used in many problems, especially when the network is shallow and the problem is sequential but does not depend too much on data from far back in time. For many RNN applications, the numerical instabilities (if any) are also still treatable using the approaches discussed above.

3. PyTorch Implementation of RNN to Approximate a Hypothetical Reservoir Flood Routing

Similar to the approach we took in Module 3, we will demonstrate the PyTorch implementation of RNN for a hypothetical engineering problem with a legitimate mathematical system. The reason for working with a hypothetical problem is so that we can have a clean case where everything is clear and traceable for the sake of discussion. However, the code will be general enough to be implemented later for a real-world problem with minimum tweaks (if needed), as far as a plain vanilla RNN is concerned. In this module, the hypothetical problem chosen is the modelling of reservoir flood routing, as detailed next.

3.1 RNN as surrogate model for level-pool routing

In civil engineering, especially in hydrology, routing the river flow (flood routing) and predicting flood propagation are very important activities. Water levels and discharges are gauged at various stations along a river, providing historical flow data for analysis. There are various methods employed for flood-routing analysis, among others are the Muskingum method for river/channel routing and level-pool routing for reservoirs. More detailed hydraulic analysis may involve numerically solving the Saint-Venant equations using numerical methods such as finite difference, finite volume and finite element methods. There are various inputs and parameters to be considered depending on the governing equations and routing method employed. The major ones may include inflow and outflow hydrographs, river geometry (length, cross-sections, meandering, etc.), rainfall, lateral flow, Manning's roughness coefficient, infiltration, subsurface flow, storage and gate operations.

Relevant to RNN as a surrogate model for flood routing, we can try to establish the relationship between the inflow and outflow hydrographs, as these are sequential in nature. For now, other parameters and properties of the system (i.e. reservoir geometry and properties, storage, gate

operation, etc.) are not explicitly considered as inputs. Instead, these are left as a “black box”, where the RNN uses its hidden states to learn and carry forward the information needed to establish the relationship between the inflow and outflow hydrographs. Later, once we get more advanced with the RNN discussion and technology, these parameters can be explicitly considered as additional inputs (features) to the model.

Similar to the strategy in Module 3 where we used legitimate but hypothetical beam’s matrix system, herein, we will generate labelled data of inflow (I) and outflow (O) hydrographs of a hypothetical reservoir using the level-pool routing equations given below (MSMA, 2012):

Continuity equation:

$$\frac{dS}{dt} = I - O \quad (29)$$

Storage-outflow relationship:

$$S = f(O) \text{ for monotonic relationship} \quad (30)$$

where S is the reservoir storage.

For the generation of the labelled data, a hypothetical reservoir with fixed geometry and outlet characteristics is considered. A total of 5000 different inflow hydrographs is then generated by varying their peak discharge, I_p , time to peak, t_p and duration, T within reasonable ranges. An idealized triangular shape of the inflow hydrograph has been used for this purpose, with its typical representation shown in Figure 14.

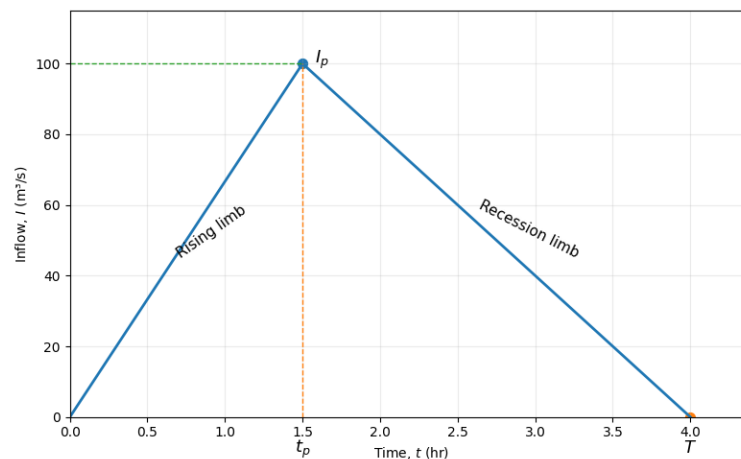


Figure 14: A typical idealized triangular inflow hydrograph

Each inflow hydrograph is then routed through the same hypothetical reservoir to obtain its corresponding outflow hydrograph. Thus, the generated inflow and outflow hydrographs form the labelled input-target data for training the RNN. For those interested in the details of the data generation, refer to Table 1 in the Appendix.

The generated data can be downloaded from:

https://github.com/msnm-official/ai_modules/blob/d9b6d36639d7717fbdd1993ba1c9153f629a073d/reservoir_rnn_5000_samples.csv

Next, we will discuss the steps to build the surrogate model, as summarized in Figure 15.

3.2 Step-by-step implementation

Step 1: Import libraries

The main libraries that we will use are:

- NumPy (for numerical and array operations)
- Pandas (for reading and handling our dataset)
- PyTorch (for building and training the RNN)
- Matplotlib (for plotting and visualization)
- Scikit-learn (for data preprocessing and other ML procedures)

Copyable code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

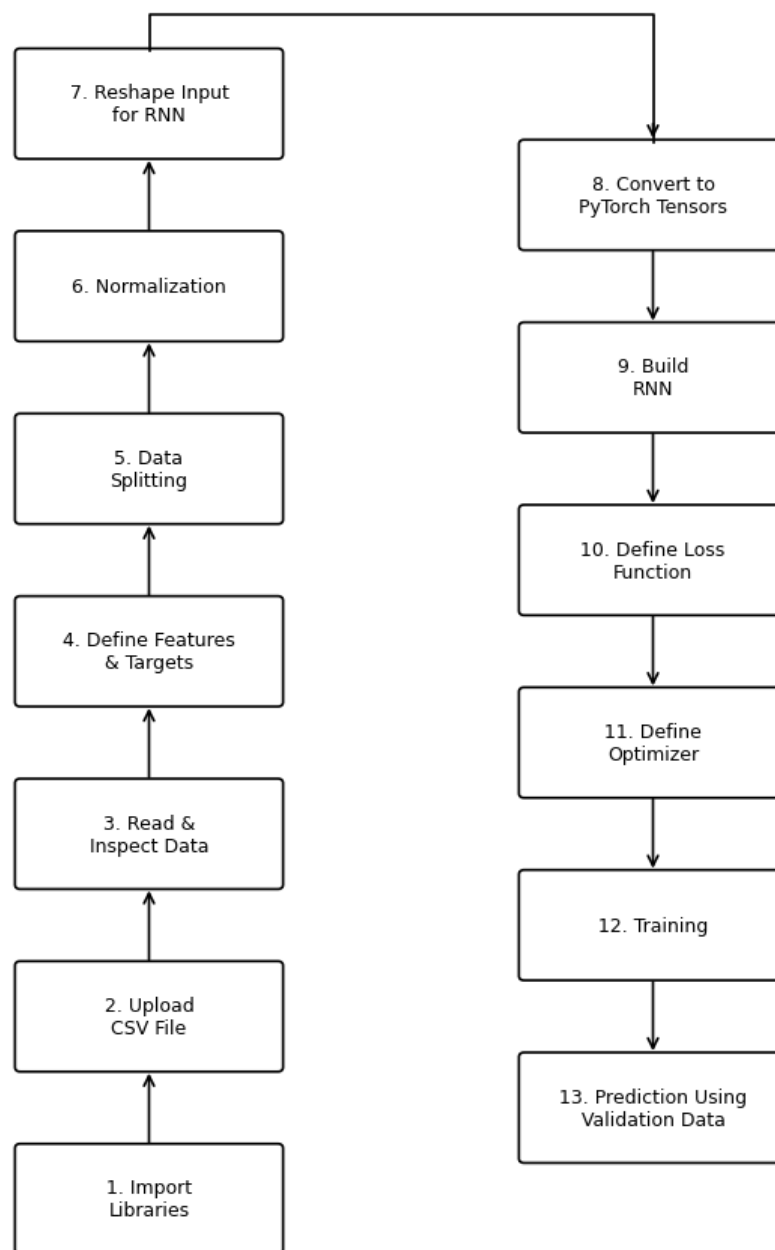
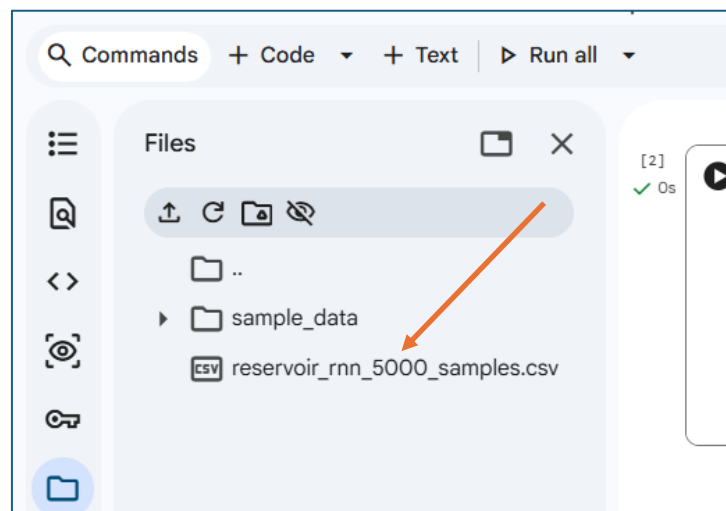


Figure 15: RNN surrogate model workflow for level-pool routing

Step 2: Upload CSV file

Before the data can be read and processed by our Python code, we first must upload the CSV file into our Google Colab. To do this, click the Files icon (folder icon) on the left panel of Google Colab. Then, click the Upload icon and select the file `reservoir_rnn_5000_samples.csv` from your computer. Once uploaded, the file should appear in the Files panel. The CSV file is now available in the current Colab session and can be read by our Python code in the next step.



Step 3: Read and inspect data

To read a CSV file, we use the Pandas function `pd.read_csv()`. To inspect the data, we request to see the heading of the table in the DataFrame and the first 5 rows using the function `print(data.head())`. We can also request the size of the data using function `print("\nDataset shape:", data.shape)`.

Copyable code:

```
data = pd.read_csv('reservoir_rnn_5000_samples.csv')

print(data.head())

print("\nDataset shape:", data.shape)
```

If we run the code, we will see the output below. As we can see, they are 25 columns. Note that the numbering shown at the far left of the DataFrame (i.e., 0, 1, 2, ...) is the Pandas DataFrame index and is not part of the 25 columns.

The first column, `sample_id`, is just the numbering of each sample. The 2nd to 4th columns describe the parameters used to generate the inflow hydrographs (as shown in Figure 14), whilst the 5th column is the maximum reservoir stage calculated from the level-pool routing. The main data for our model are the remaining columns, as they are the input and output samples for our training. The 6th to 15th columns are the discretized inflow values (i.e., $\{I_1, I_2, \dots, I_{10}\}$), whilst the 16th to 25th columns are the corresponding outflow values calculated from the level-pool routing of Eqs. (29) and (30) (i.e., $\{O_1, O_2, \dots, O_{10}\}$).

```
[3] data = pd.read_csv('reservoir_rnn_5000_samples.csv')
print(data.head())
print("\nDataset shape:", data.shape)
```

	sample_id	peak_inflow_m3s	time_to_peak_hr	duration_hr	max_stage_m	I_1 \
0	1	99.656044	1.272686	4.081093	2.188571	0.0
1	2	69.499060	1.828660	4.066857	1.737628	0.0
2	3	107.273813	1.958577	3.304043	2.320059	0.0
3	4	92.763123	1.208295	3.054983	2.038504	0.0
4	5	38.475961	1.054823	3.455699	1.152748	0.0

	I_2	I_3	I_4	I_5	...	O_1	O_2 \
0	39.151844	78.303687	91.589844	73.847388	...	0.0	11.351680
1	19.002733	38.005465	57.008198	64.178708	...	0.0	4.490686
2	27.385653	54.771305	82.156958	103.971154	...	0.0	7.209507
3	38.385952	76.771905	78.110159	52.994079	...	0.0	11.073067
4	18.238115	36.476230	31.341642	23.328743	...	0.0	4.255627

	O_3	O_4	O_5	O_6	O_7	O_8 \
0	50.280964	81.115290	82.562192	66.777694	49.529841	32.367193
1	22.075412	43.334421	58.408397	56.650492	42.908984	28.090149
2	33.633987	63.890607	90.113283	84.612728	48.624313	18.144295
3	49.187895	74.217690	66.466873	43.601154	20.052094	6.282224
4	21.041541	31.560357	28.052964	20.971122	13.490953	6.498811

	O_9	O_10
0	15.735542	5.523020
1	13.630430	4.841144
2	5.055090	2.096239
3	2.456622	1.209561
4	2.517380	1.232961

[5 rows x 25 columns]
Dataset shape: (5000, 25)

Note that, for columns I_1 and O_1 , the entries are always zero because they represent the prescribed initial inflow and outflow conditions, taken as zero for all samples.

Step 4: Define features and targets

We will use the discretized inflow hydrograph values as the input features and the corresponding outflow hydrograph values as the target outputs. Therefore, the columns $I_1, I_2, \dots, I_{10}$ are selected as the input data, whilst $O_1, O_2, \dots, O_{10}$ are selected as the target data.

We then convert the selected columns into NumPy arrays using `.values` so that they can be processed in the subsequent steps.

Copyable code:

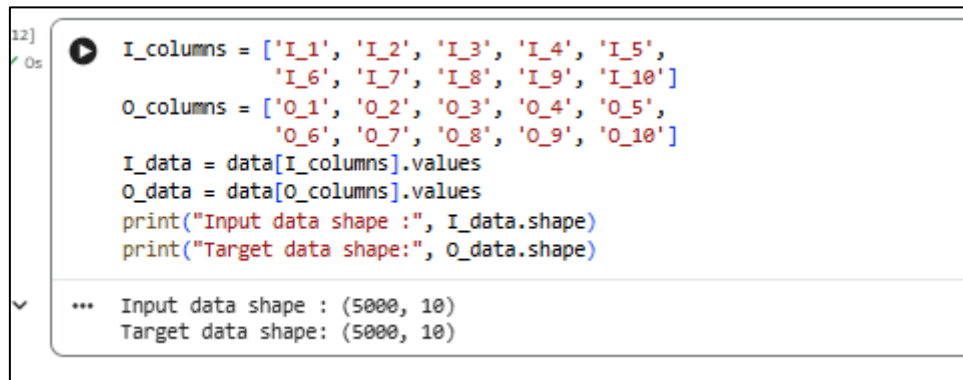
```
I_columns = ['I_1', 'I_2', 'I_3', 'I_4', 'I_5',
             'I_6', 'I_7', 'I_8', 'I_9', 'I_10']
O_columns = ['O_1', 'O_2', 'O_3', 'O_4', 'O_5',
             'O_6', 'O_7', 'O_8', 'O_9', 'O_10']
I_data = data[I_columns].values
```

```
O_data = data[O_columns].values

print("Input data shape :", I_data.shape)

print("Target data shape:", O_data.shape)
```

If we run the code, we will see this output:

A screenshot of a Jupyter Notebook cell. The code defines two lists of column names, I_columns and O_columns, and then extracts the corresponding data from a 'data' object into I_data and O_data arrays. It then prints the shapes of these arrays. The output shows that I_data has a shape of (5000, 10) and O_data has a shape of (5000, 10).

```
12] In [ ]: I_columns = ['I_1', 'I_2', 'I_3', 'I_4', 'I_5',
                        'I_6', 'I_7', 'I_8', 'I_9', 'I_10']
          O_columns = ['O_1', 'O_2', 'O_3', 'O_4', 'O_5',
                        'O_6', 'O_7', 'O_8', 'O_9', 'O_10']
          I_data = data[I_columns].values
          O_data = data[O_columns].values
          print("Input data shape :", I_data.shape)
          print("Target data shape:", O_data.shape)

Out[ ]: ... Input data shape : (5000, 10)
        Target data shape: (5000, 10)
```

From the printed shapes, we can see that the NumPy arrays have been created and their shapes are correct.

Step 5: Data splitting

We split all the data into training sets (70%) and validation sets (30%). As mentioned in previous modules, we use such a splitting because we are not concerning ourselves with hyperparameter tuning for now.

Copyable code:

```
split = int(0.7 * len(I_data))

I_train = I_data[:split]
O_train = O_data[:split]

I_val = I_data[split:]
O_val = O_data[split:]

print("Training input shape :", I_train.shape)
print("Training target shape:", O_train.shape)
print("Validation input shape :", I_val.shape)
print("Validation target shape:", O_val.shape)
```

If we run the code, we will see the output below. From the printed shapes, we can see that the splitting has been successful, where the sample sizes are appropriate.

```
[17]
✓ Os
split = int(0.7 * len(I_data))
I_train = I_data[:split]
O_train = O_data[:split]
I_val = I_data[split:]
O_val = O_data[split:]

print("Training input shape :", I_train.shape)
print("Training target shape:", O_train.shape)
print("Validation input shape :", I_val.shape)
print("Validation target shape:", O_val.shape)

Training input shape : (3500, 10)
Training target shape: (3500, 10)
Validation input shape : (1500, 10)
Validation target shape: (1500, 10)
```

Step 6: Normalization

Essentially, there is no difference in the procedure and the code from those given in the previous modules, where the process is automated using the MinMaxScaler class. The maximum and minimum values are still taken from the training sets, hence the use of fit_transform() for I_train, and O_train, and transform() for I_val, and O_val.

Copyable code:

```
scaler_I = MinMaxScaler()
scaler_O = MinMaxScaler()
I_train_norm = scaler_I.fit_transform(I_train)
I_val_norm = scaler_I.transform(I_val)
O_train_norm = scaler_O.fit_transform(O_train)
O_val_norm = scaler_O.transform(O_val)

print("Before normalization:")
print("I_train[0] =", I_train[0])
print("\nAfter normalization:")
print("I_train_norm[0] =", I_train_norm[0])
```

If we run the code, we will see the output below. From the printed values, we can see that the normalization has been successful, where none of the values of I_train_norm are below zero or exceed one.

```
21] scaler_I = MinMaxScaler()
✓ 0s scaler_O = MinMaxScaler()
I_train_norm = scaler_I.fit_transform(I_train)
I_val_norm = scaler_I.transform(I_val)
O_train_norm = scaler_O.fit_transform(O_train)
O_val_norm = scaler_O.transform(O_val)
print("Before normalization:")
print("I_train[0] =", I_train[0])
print("\nAfter normalization:")
print("I_train_norm[0] =", I_train_norm[0])

Before normalization:
I_train[0] = [ 0.          39.15184371  78.30368742  91.58984405  73.84738786  56.10493167
 38.36247548  20.62001929  2.8775631  0.          ]

After normalization:
I_train_norm[0] = [0.          0.61515186  0.61515186  0.72225868  0.55625403  0.49679123
 0.49521242  0.40080278  0.11316821  0.          ]
```

Step 7: Reshape input for RNN

For the RNN, the input data must be arranged according to its sequential structure. PyTorch RNN expects the input in the following format:

(number of samples, number of time steps, number of features)

In our case, each sample consists of 10 sequential inflow values, $(I_1, I_2, \dots, I_{10})$, and there is only one feature (input) at each time step, which is the inflow. Thus, for our problem, the training input would be in the form of (3500, 10, 1), reshaped from (3500, 10). Similarly, the validation input is reshaped from (1500, 10) to (1500, 10, 1).

Copyable code:

```
I_train_rnn = I_train_norm.reshape(3500, 10, 1)
I_val_rnn = I_val_norm.reshape(1500, 10, 1)
print("I_train_rnn shape =", I_train_rnn.shape)
print("I_val_rnn shape =", I_val_rnn.shape)
```

If we run the code, we will see this output:

```
22] I_train_rnn = I_train_norm.reshape(3500, 10, 1)
✓ 0s I_val_rnn = I_val_norm.reshape(1500, 10, 1)
print("I_train_rnn shape =", I_train_rnn.shape)
print("I_val_rnn shape =", I_val_rnn.shape)

... I_train_rnn shape = (3500, 10, 1)
I_val_rnn shape = (1500, 10, 1)
```

From the printed shapes, we can see that the `I_train_norm` and `I_val_norm` have been reshaped into the intended RNN input forms (i.e., `I_train_rnn`, `I_val_rnn`).

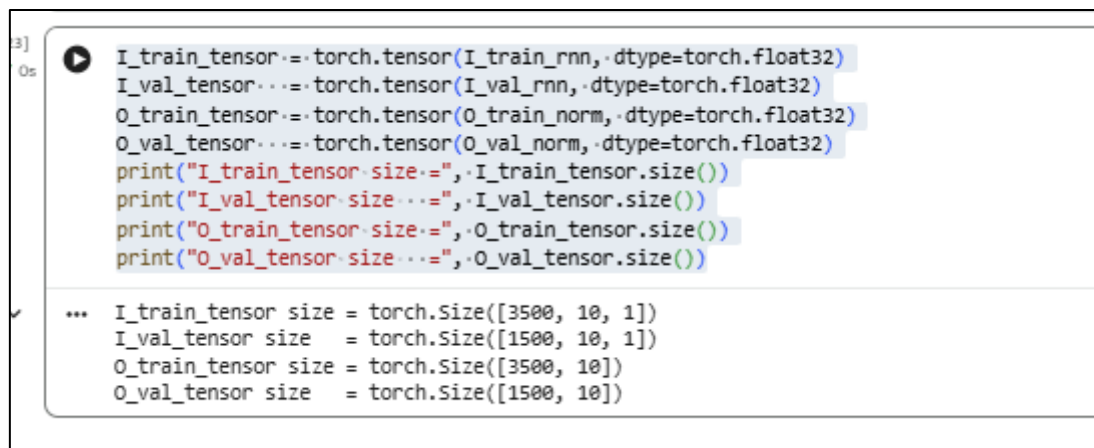
Step 8: Convert to PyTorch tensors

However, since we have decided to use PyTorch to build our RNN architecture, we must convert our data into a format that can be read and handled by PyTorch. For this, we use the PyTorch function `torch.tensor()`. The resulting data are called PyTorch tensors. A tensor can be considered as a data structure used by PyTorch to store and perform operations on numerical data.

Copyable code:

```
I_train_tensor = torch.tensor(I_train_rnn, dtype=torch.float32)
I_val_tensor = torch.tensor(I_val_rnn, dtype=torch.float32)
O_train_tensor = torch.tensor(O_train_norm, dtype=torch.float32)
O_val_tensor = torch.tensor(O_val_norm, dtype=torch.float32)
print("I_train_tensor size =", I_train_tensor.size())
print("I_val_tensor size =", I_val_tensor.size())
print("O_train_tensor size =", O_train_tensor.size())
print("O_val_tensor size =", O_val_tensor.size())
```

If we run the code, we will see this output:



```
3] In [10]: I_train_tensor = torch.tensor(I_train_rnn, dtype=torch.float32)
          I_val_tensor = torch.tensor(I_val_rnn, dtype=torch.float32)
          O_train_tensor = torch.tensor(O_train_norm, dtype=torch.float32)
          O_val_tensor = torch.tensor(O_val_norm, dtype=torch.float32)
          print("I_train_tensor size =", I_train_tensor.size())
          print("I_val_tensor size =", I_val_tensor.size())
          print("O_train_tensor size =", O_train_tensor.size())
          print("O_val_tensor size =", O_val_tensor.size())

Out[10]: ... I_train_tensor size = torch.Size([3500, 10, 1])
          I_val_tensor size = torch.Size([1500, 10, 1])
          O_train_tensor size = torch.Size([3500, 10])
          O_val_tensor size = torch.Size([1500, 10])
```

From the printed sizes, we can see that the conversion has been successful, where the sample sizes are as expected.

Step 9: Build RNN architecture

This is the stage where we build our RNN surrogate model. To build the RNN, we use class `RNN(nn.Module)`. Within this class, we use `nn.RNN()`, which is a PyTorch class that creates the recurrent layer to carry out the sequential operations described earlier.

For our problem, there is only one input feature at each time step, which is the inflow, I_t , thus `input_size=1`. In our earlier formulation, we used only two neurons in the hidden layer so that the RNN calculations could be easily demonstrated and traced by hand. In the actual implementation here, we increase the number to 32 hidden neurons to provide the RNN with greater capacity to learn the relationship between the inflow and outflow hydrographs, thus `hidden_size=32`. The underlying RNN formulation remains the same regardless of the number of hidden neurons used. We also use `batch_first=True` so that the RNN accepts the input in the form discussed in Step 7.

We then define the forward passing of the input. The complete inflow sequence is first passed through the RNN using `x, h = self.rnn(x)`. The resulting hidden states are then passed through the linear output layer using `x = self.fc(x)`. Since the linear layer initially produces an output of shape (number of samples, 10, 1), we use `squeeze(-1)` to remove the last dimension, giving the final output in the form (number of samples, 10), which is the same form as our target data.

Copyable code:

```
class RNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.rnn = nn.RNN(
            input_size=1,
            hidden_size=32,
            batch_first=True,
            nonlinearity='tanh'
        )

        self.fc = nn.Linear(32, 1)
    def forward(self, x):
        x, h = self.rnn(x)
        x = self.fc(x)
        x = x.squeeze(-1)

        return x
model = RNN()
print(model)
```

If we run the code, we will see this input:

```
10] ▶ class RNN(nn.Module):
    0s
    def __init__(self):
        super().__init__()
        self.rnn = nn.RNN(
            input_size=1,
            hidden_size=32,
            batch_first=True,
            nonlinearity='tanh'
        )

        self.fc = nn.Linear(32, 1)
    def forward(self, x):
        x, h = self.rnn(x)
        x = self.fc(x)
        x = x.squeeze(-1)

        return x
model = RNN()
print(model)

... RNN(
  (rnn): RNN(1, 32, batch_first=True)
  (fc): Linear(in_features=32, out_features=1, bias=True)
)
```

From the printed output, we can see that our RNN architecture has been successfully generated.

Step 10: Define loss function

Since our problem is still a regression problem, there is no change to the choice of loss function compared with previous modules. We continue to use the Mean Squared Error (MSE) by employing `criterion = nn.MSELoss()`.

Copyable code:

```
criterion = nn.MSELoss()
print("Loss function:", criterion)
```

If we run the code, we will see this output:

```
12] ▶ criterion = nn.MSELoss()
    0s
    print("Loss function:", criterion)

... Loss function: MSELoss()
```

From the printed output, we can see that MSE has been set up as the loss function.

Step 11: Define optimizer

Similar to the loss function, there is no change to the optimizer compared with the previous module. We continue to use the Adam optimizer with a learning rate of 0.001. The `model.parameters()` passes all the learnable parameters of our RNN model to the optimizer, which will then update their values during the training process.

Copyable code:

```
learning_rate = 0.001
optimizer = optim.Adam(
    model.parameters(),
    lr=learning_rate)
```

Run the code, but don't expect any output because we do not ask the code to print or display anything.

Step 12: Training

Essentially, there is no change to the training procedure given in the previous modules. To execute the training means we need to instruct the following process:

- i. the forward pass
- ii. the loss calculation
- iii. the loss gradient calculation
- iv. the parameter update

The model now receives the sequential input, `I_train_tensor`, and predicts the corresponding outflow sequence, `O` (i.e., `O_pred`). The loss is then calculated from the error between `O_pred` and `O_train_tensor`. The rest of the lines remain as in the previous modules.

However, for the RNN, the loss gradient calculation automatically involves the backpropagation through time (BPTT) process discussed earlier. PyTorch carries out this process automatically when we use `loss.backward()`.

Copyable code:

```
from IPython.display import clear_output
epochs = 500
loss_history = []
for epoch in range(epochs):
    O_pred = model(I_train_tensor) # Forward pass
    loss = criterion(
        O_pred,
        O_train_tensor
    ) # Loss calculation
    optimizer.zero_grad() # Loss gradient calculation
    loss.backward()       # BPTT
```

```
optimizer.step()    # Parameter update
loss_history.append(loss.item()) # Record loss
# Live plot
if (epoch + 1) % 100 == 0:
    clear_output(wait=True)
    plt.figure(figsize=(7, 4))
    plt.plot(loss_history)
    plt.xlabel("Epoch")
    plt.ylabel("MSE Loss")
    plt.title("RNN Surrogate Training Loss")
    plt.grid(True)
    plt.show()
    print(
        f"Epoch [{epoch+1}/{epochs}], "
        f"Loss: {loss.item():.6f}"
    )
```

By running the code, we would obtain the convergence curve as shown in Figure 16. As can be seen, the loss reduces as the number of epochs increases. This indicates that loss is being minimized and the errors are diminishing, hence showing convergence.

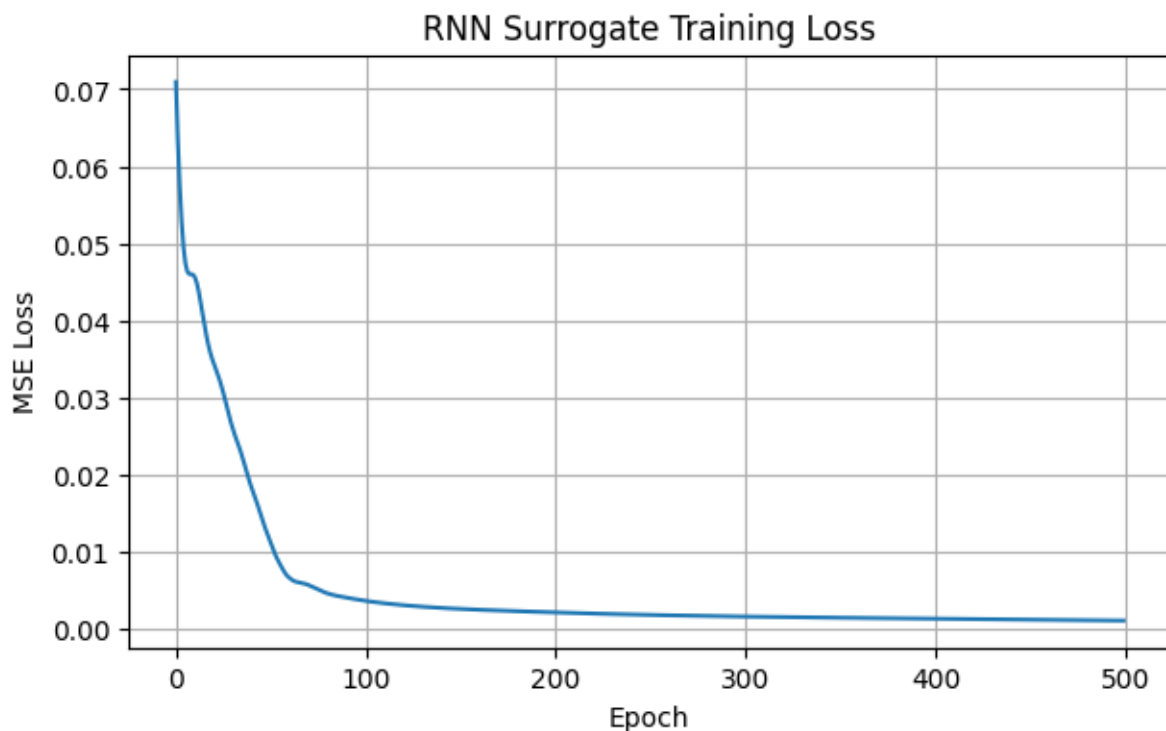


Figure 16: Convergence of the Loss

Step 13: Prediction using validation data

Now that we have completed the training, it is time to check whether our model can really make predictions. For this, we will use the remaining 30% of the dataset, which we have specified as `I_val` and later transformed into a PyTorch tensor, denoted as `I_val_tensor`. It will be used to produce the predicted values, `O_val_pred_norm`, which are later denormalized or inverse-transformed into `O_val_pred`.

To make the prediction, we use `model.eval()` together with `torch.no_grad()` to tell PyTorch that we are not training anymore (thus, we do not need the gradients).

We do not need to inverse-transform `O_val` because it is already in its original magnitude. So, the comparison will be `O_val_pred` vs `O_val`.

Copyable code:

```
model.eval()
with torch.no_grad():
    O_val_pred_norm = model(
        I_val_tensor
    )
O_val_pred_norm = O_val_pred_norm.numpy()
O_val_pred = scaler_O.inverse_transform(
    O_val_pred_norm
)
```

The code above calculates the predicted values of the outputs, but we are yet to compare them against the actual values. Similar to Module 3, we can create the predicted-actual pairs. However, instead of the six DOFs, we now have the outflow at the 10 time steps.

Copyable code 2: Creating the predicted-actual pairs for the 10 outputs

```
actual_O1 = O_val[:, 0]
predicted_O1 = O_val_pred[:, 0]
actual_O2 = O_val[:, 1]
predicted_O2 = O_val_pred[:, 1]
actual_O3 = O_val[:, 2]
predicted_O3 = O_val_pred[:, 2]
actual_O4 = O_val[:, 3]
predicted_O4 = O_val_pred[:, 3]
actual_O5 = O_val[:, 4]
predicted_O5 = O_val_pred[:, 4]
actual_O6 = O_val[:, 5]
predicted_O6 = O_val_pred[:, 5]
actual_O7 = O_val[:, 6]
predicted_O7 = O_val_pred[:, 6]
actual_O8 = O_val[:, 7]
predicted_O8 = O_val_pred[:, 7]
actual_O9 = O_val[:, 8]
predicted_O9 = O_val_pred[:, 8]
actual_O10 = O_val[:, 9]
predicted_O10 = O_val_pred[:, 9]
```

Modules for Machine and Deep Learning (Module 4: RNN and a surrogate model for Flood Routing with PyTorch)

To assess the performance of the model, we can either calculate the difference between O_{val_pred} vs O_{val} (in the form of performance metrics, MSE, RMSE, MAE and R^2) or create a scatter plot where the coordinates of each data point are (O_{val} , O_{val_pred}) and assess how far the points are from the 45-degree reference line.

A quick check on the model performance can be made by looking at the MAE and RMSE, where relatively small values indicate a better-performing model. For R^2 , a value closer to 1 indicates a better agreement between the actual and predicted outflow values. Also, for a good-performing model, the scattered data points should be very close or in proximity to the 45-degree reference line. Since our outputs are sequential in nature, we can further assess the model by plotting the actual and predicted outflow hydrographs against time to see how closely the RNN reproduces the shape of the outflow hydrograph.

Copyable code 3: Calculating and printing the performance

```
for i in range(10):
    actual = O_val[:, i]
    predicted = O_val_pred[:, i]
    MSE = mean_squared_error(actual, predicted)
    RMSE = np.sqrt(MSE)
    MAE = mean_absolute_error(actual, predicted)
    R2 = r2_score(actual, predicted)
    print(f"Performance for output O{i+1}")
    print(f"MSE = {MSE:.8e}")
    print(f"RMSE = {RMSE:.8e}")
    print(f"MAE = {MAE:.8e}")
    print(f"R2 = {R2:.6f}")
    print()
```

If we run all the codes above, we will see the following output:

<pre>71 for i in range(10): 72 actual = O_val[:, i] 73 predicted = O_val_pred[:, i] 74 MSE = mean_squared_error(actual, predicted) 75 RMSE = np.sqrt(MSE) 76 MAE = mean_absolute_error(actual, predicted) 77 R2 = r2_score(actual, predicted) 78 print(f"Performance for output O{i+1}") 79 print(f"MSE = {MSE:.8e}") 80 print(f"RMSE = {RMSE:.8e}") 81 print(f"MAE = {MAE:.8e}") 82 print(f"R² = {R2:.6f}") 83 print() 84 ... 85 RMSE = 1.41888857e-04 86 MAE = 1.41888857e-04 87 R² = 0.000000 88 89 Performance for output O2 90 MSE = 2.04781923e+00 91 RMSE = 1.43102035e+00 92 MAE = 1.11462246e+00 93 R² = 0.816383</pre>	<pre>Performance for output O3 MSE = 2.47679466e-01 RMSE = 4.97674056e-01 MAE = 3.77270843e-01 R² = 0.998716 Performance for output O4 MSE = 2.65428538e+00 RMSE = 1.62919777e+00 MAE = 1.37990624e+00 R² = 0.993577 Performance for output O5 MSE = 3.68833006e+00 RMSE = 1.92050255e+00 MAE = 1.51639747e+00 R² = 0.992073 Performance for output O6 MSE = 3.47118555e+00 RMSE = 1.86311179e+00 MAE = 1.45923718e+00 R² = 0.991837</pre>	<pre>Performance for output O7 MSE = 1.73990897e+00 RMSE = 1.31905609e+00 MAE = 1.07538463e+00 R² = 0.993549 Performance for output O8 MSE = 2.73724677e+00 RMSE = 1.65446268e+00 MAE = 1.41702718e+00 R² = 0.981583 Performance for output O9 MSE = 2.14651781e+00 RMSE = 1.4650993e+00 MAE = 9.65043526e-01 R² = 0.960411 Performance for output O10 MSE = 6.46889122e-01 RMSE = 8.04294176e-01 MAE = 5.62508983e-01 R² = 0.927405</pre>
---	---	---

From the output, we can see the performance metrics for the outflow values at the different time steps. As mentioned, a quick check on the model performance can be made by looking at the MAE, RMSE and R^2 , where relatively small values of the former two or values close to 1 for the latter indicate a good-performing model.

Based on the values, especially the R^2 ranging from 0.816 to 0.998 for O_2 to O_{10} , respectively, we can assess how well our RNN model is able to predict the outflow hydrograph, even before we carry out any hyperparameter tuning.

Note that, the R^2 value for O_1 is excluded from the assessment since O_1 represents the prescribed initial outflow and has a constant value of zero for all samples, as mentioned in Step 3.

Next, we can visually assess the performance of the model by plotting the scatter plots.

Copyable code 4: Scatter plots

```
for i in range(1, 10):
    actual = O_val[:, i]
    predicted = O_val_pred[:, i]
    R2 = r2_score(
        actual,
        predicted
    )
    min_val = min(
        actual.min(),
        predicted.min()
    )
    max_val = max(
        actual.max(),
        predicted.max()
    )

    plt.figure(figsize=(6, 6))
    plt.scatter(
        actual,
        predicted,
        alpha=0.5
    )
    # 45-degree reference line
    plt.plot(
        [min_val, max_val],
        [min_val, max_val],
        '--',
        label='45° Reference Line'
    )
    plt.xlabel(f"Actual O{i+1} (m3/s)")
    plt.ylabel(f"Predicted O{i+1} (m3/s)")
    plt.title(
        f"Actual vs Predicted O{i+1}\n"
        f" $R^2 = \{R2:.4f\}$ "
    )
    plt.legend()
    plt.grid(True)
    plt.axis('equal')
    plt.show()
```

Modules for Machine and Deep Learning (Module 4: RNN and a surrogate model for Flood Routing with PyTorch)

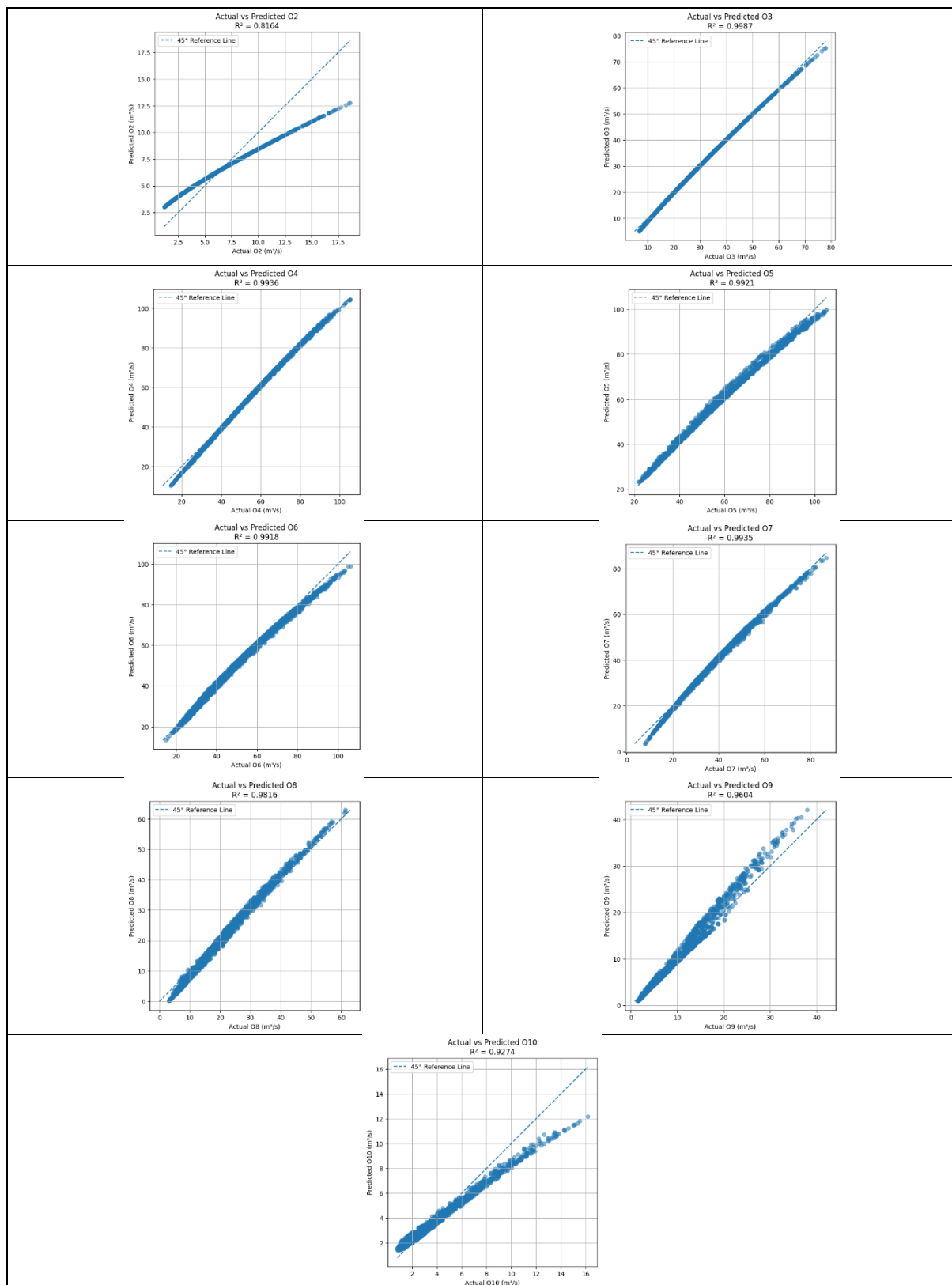


Figure 17: Scatter plots for the predictions of the outflow hydrographs at 10 time intervals

If we run the code above, we would obtain the scatter plots shown in Figure 17. Based on the figure, we can see that our model is able to learn and approximate the level-pool routing equations represented by Eqs. (29) and (30), despite some apparent divergences for O_2 and O_{10} .

The divergence at O_2 may partly be attributed to its position near the beginning of the sequence, immediately after the prescribed initial condition $O_1 = 0$, where the RNN has only limited preceding information carried through its hidden state. Meanwhile, the larger scatter observed at O_{10} occurs towards the end of the hydrograph, where the flow is generally within the recession period.

These discrepancies may also be associated with the relatively simple plain vanilla RNN architecture employed in this example. Furthermore, no hyperparameter tuning has been carried out, as our main intention in this module is to demonstrate the fundamental formulation and implementation of an RNN. The prediction performance could potentially be improved through hyperparameter tuning, such as the number of hidden neurons, learning rate and number of training epochs.

Copyable code 5: Actual and predicted outflow hydrographs

In addition to the scatter plots, since the outputs in our problem represent sequential outflow values, we can further assess the performance of the RNN by plotting the actual and predicted outflow hydrographs against time. This allows us to visually assess whether the model is able to reproduce not only the magnitude of the outflow, but also the overall shape and temporal behaviour of the hydrograph.

In this code, we select 4 samples to be plotted.

```
# Time corresponding to the 10 time steps
time = np.arange(0, 5.0, 0.5)

# Randomly select 4 validation samples
sample_indices = np.random.choice(
    len(O_val),
    size=4,
    replace=False
)

# Plot the selected validation samples
for i in sample_indices:

    plt.figure(figsize=(7, 5))

    plt.plot(
        time,
        O_val[i],
        'o-',
        label='Actual Outflow'
    )
```

```
plt.plot(
    time,
    O_val_pred[i],
    's--',
    label='Predicted Outflow'
)

plt.xlabel("Time (hr)")
plt.ylabel("Outflow (m3/s)")

plt.title(
    f"Actual vs Predicted Outflow Hydrograph "
    f"(Validation Sample {i+1})"
)
plt.legend()
plt.grid(True)
plt.show()
```

Figure 18 shows the outflow hydrographs of four randomly selected validation samples. As can be seen, the predicted hydrographs are very close to the actual hydrographs, indicating that the RNN model is able to successfully learn and approximate the level-pool routing relationship represented by Eqs. (29) and (30).

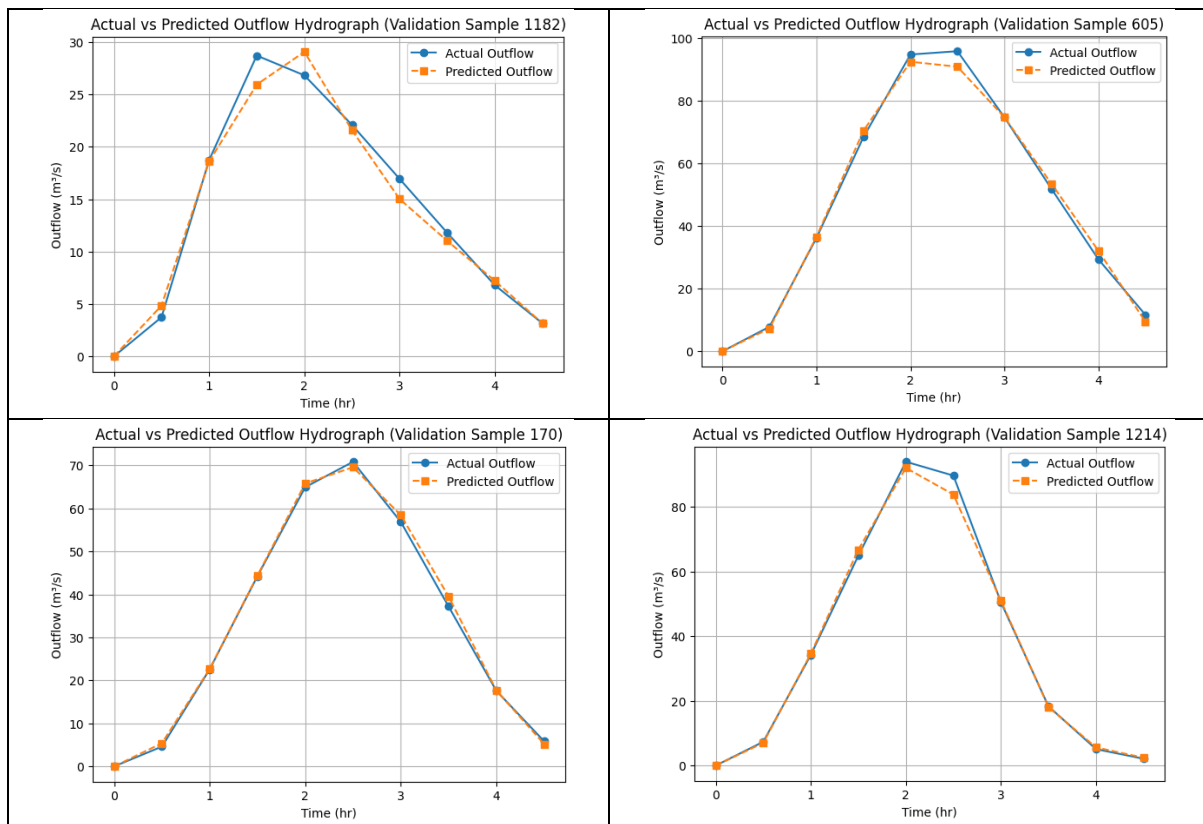


Figure 18: Actual and predicted outflow hydrographs for four randomly selected validation samples

Reference:

Department of Irrigation and Drainage Malaysia. (2012). *Urban Stormwater Management Manual for Malaysia (MSMA), 2nd Edition*. Kuala Lumpur, Malaysia: Department of Irrigation and Drainage Malaysia. (https://www.water.gov.my/jps/resources/MSMA2ndEdition_august_2012.pdf)

Appendix:

Note: The reservoir geometry, outlet parameters and inflow hydrograph ranges adopted in Table 1 represent hypothetical but reasonable values for the purpose of demonstrating the level-pool routing procedure. They should not be interpreted as values prescribed by MSMA.

[24] `# Display the table`
`display(Table_1)`

	Item	Adopted value/range	Unit	Description
0	Number of samples	5000	–	Total number of labelled samples generated
1	Number of time steps	10	–	Number of discretized inflow and outflow value...
2	Time interval, Δt	30	min	Routing time interval
3	Total sequence period	4.5	hr	Sequence from $t = 0$ to $t = 4.5$ hr
4	Initial reservoir stage, H_i	0	m	Prescribed initial condition
5	Initial outflow, O_i	0	m ³ /s	Prescribed initial condition
6	Reservoir surface area	$A(H) = 40,000 + 10,000H$	m ²	Hypothetical stage-area relationship
7	Reservoir storage	$S(H) = 40,000H + 5,000H^2$	m ³	Stage-storage relationship obtained from the a...
8	Weir coefficient, C_w	1.70	–	Coefficient adopted for the uncontrolled outlet
9	Weir length, L_w	15	m	Length of the uncontrolled outlet
10	Reservoir outflow	$O(H) = 1.70(15)H^{3/2}$	m ³ /s	Adopted stage-outflow relationship
11	Peak inflow, I_p	30–120	m ³ /s	Randomly varied for the generation of differen...
12	Time to peak, t_p	1.00–2.25	hr	Randomly varied
13	Hydrograph duration, T	3.00–4.50	hr	Randomly varied
14	Inflow hydrograph shape	Triangular	–	Idealized piecewise-linear inflow hydrograph
15	Routing method	Level-pool (Modified Puls)	–	Used to calculate the corresponding outflow hy...
16	Routing equation	$S_2/\Delta t + O_2/2 = S_1/\Delta t - O_1/2 + (I_1 + I_2)/2$	–	Storage-indicator form used for the routing ca...